

CS324 Deep Learning - Assignment 1

1. Part I: The perceptron

Two two-dimensional Gaussian distributions were sampled with 100 points per class. For each class, 80 points were used for training and 20 points were used for testing. The labels were encoded as -1 and +1, and the bias was implemented as an additional constant input.

The perceptron was trained with the batch perceptron gradient described in the assignment tutorial. The results were:

Experiment	Training accuracy	Test accuracy
Well-separated means, variance 0.25	100.00%	100.00%
Close means, variance 0.25	90.62%	90.00%
Well-separated means, variance 2.0	93.75%	87.50%

When the means are close, the two distributions overlap more strongly. A single linear decision boundary cannot separate all samples, so the training error does not reach zero. Increasing the variance also increases the overlap between the classes, which reduces especially the test accuracy.

2. Part II: The multi-layer perceptron

The dataset was generated with `make_moons` using 1,000 two-dimensional samples and noise 0.2. The labels were converted to two-dimensional one-hot vectors. The data was randomly split into 800 training samples and 200 test samples.

The default network has the following structure:

`Linear(2, 20) -> ReLU -> Linear(20, 2) -> SoftMax`

The implementation uses NumPy for all forward and backward computations. Softmax uses the max trick for numerical stability. Cross entropy is computed as the mean over the batch, and the combined softmax-cross-entropy gradient is `(prediction - target) / batch_size`.

With the default learning rate 0.01, hidden width 20, 1,500 epochs, and a fixed random seed of 42, the full-batch run reached approximately 87.00% test accuracy.

The notebook `Part_2/mlp_report.ipynb` plots the training and test accuracy curves as well as the corresponding loss curves.

3. Part III: stochastic and mini-batch gradient descent

The training function accepts a `batch_size` argument:

- `None` or command-line value 0: full-batch gradient descent;
- 1: stochastic gradient descent;
- any value greater than 1: mini-batch gradient descent.

Using the same dataset, architecture, learning rate, and random seed, the following 1,500-epoch results were observed:

Batch size	Training accuracy	Test accuracy	Test loss
Full batch	82.50%	87.00%	0.3566
1	97.50%	98.50%	0.0433
8	97.62%	99.00%	0.0328
16	97.12%	98.50%	0.0469
32	86.50%	94.50%	0.1878
64	86.12%	92.50%	0.2068
128	85.62%	92.00%	0.2137

In this experiment, smaller batches produced more parameter updates per epoch and reached high accuracy more quickly. Batch size 1 produced noisier updates, while batch size 8 provided a useful balance between frequent updates and stable gradients. Larger batches produced smoother but slower optimization for the selected learning rate and epoch budget.

4. How to run

From the `Part_1` directory:

```
python experiments.py
```

From the `Part_2` directory, run full-batch training:

```
python train_mlp_numpy.py --dnn_hidden_units 20 --learning_rate 0.01 --max_steps 1500 --eval
```

Run SGD:

```
python train_mlp_numpy.py --dnn_hidden_units 20 --learning_rate 0.01 --max_steps 1500 --eval
```

Run the report notebook `Part_2/mlp_report.ipynb` to reproduce the accuracy, loss, and batch-size comparison plots.